



PULSE IMAGING ENGINE

A High-Performance Medical Imaging Pipeline in C++20

Custom DICOM Ingestion · Multi-Threaded 3D Reconstruction · AVX2 SIMD Filtering · Region-Growing Anomaly Detection · Annotated Export

8-Week Engineering Build | Internship Capstone Project

Why Build This From Scratch

The brief: prove systems-level performance engineering, not just call a library

The Constraint

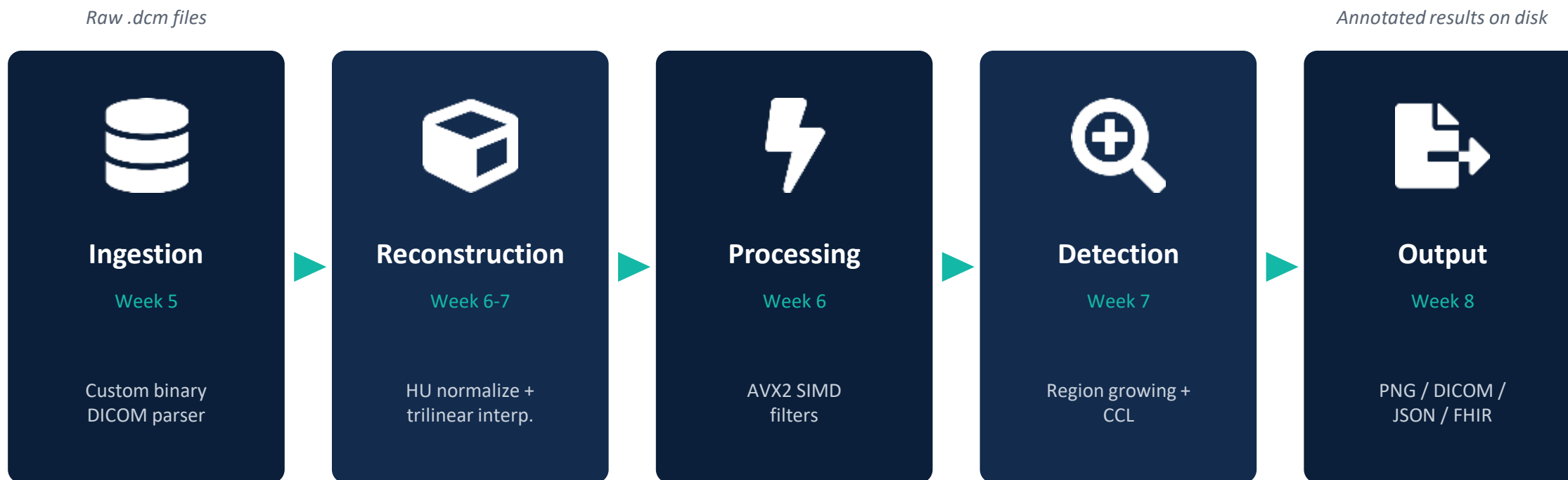
- Hand-roll the DICOM binary parser — no DCMTK in the critical path
- Hand-roll the thread pool — `std::thread` + `std::mutex` + `std::condition_variable` only
- Hand-roll SIMD filters — raw AVX2 intrinsics, not an image library
- Every layer independently correctness-verified before the next depends on it

The Result

- A working 5-layer pipeline: Ingestion → Reconstruction → Processing → Detection → Output
- Real measured AVX2 speedups — up to 15x on edge detection
- Real multi-core speedup on production hardware, verified correct, not just fast
- A real bug caught and fixed by the test suite before it reached production use

The Five-Layer Pipeline

Each layer consumes the previous layer's output type and produces the next layer's input type



Technology Stack

C++20

Language standard throughout

CMake + Conan 2

Build system & dependency management

AVX2 Intrinsics

SIMD acceleration for image filters

std::thread / mutex / condition_variable

Hand-built thread pool

libpng

PNG slice export

ThreadSanitizer

Race-condition verification

Layer 1: Ingestion

A hand-written binary DICOM Part 10 parser — zero DCMTK dependency in the runtime path



- Explicit & Implicit VR, Little & Big Endian
- Correct tag/length parsing incl. sequence skipping
- Signed and unsigned pixel representation
- Compressed transfer syntaxes rejected cleanly, by design

Real Bug Caught by Testing

A signed CT background sentinel (−2000 HU) decoded as unsigned → 63536, producing a physically impossible HU value of 62,512. Caught by sanity-checking output, fixed by honoring PixelRepresentation.

struct Slice

```
Dataset metadata;  
vector<int32_t> pixels;  
int rows, columns;  
double rescaleSlope;  
double rescaleIntercept;  
double imagePositionZ;  
double pixelSpacingRowMM;  
double pixelSpacingColMM;  
double sliceThicknessMM;
```

Layer 2: Reconstruction

Stack 2D slices into an isotropic 3D volume



The Problem

Clinical scans are rarely isotropic — e.g. $0.9\text{mm} \times 0.9\text{mm}$ in-plane pixels but 2.5mm between slices. Downstream 3D operations need uniform spacing.

The Solution

- Sort slices by Z-position from ImagePositionPatient
- Convert every voxel to Hounsfield Units using each slice's own rescale slope/intercept
- Trilinear interpolation resamples to isotropic spacing

Correctness Proof

Trilinear interpolation is mathematically exact on linear data. We reconstructed a synthetic volume with a known linear HU gradient — output matched the analytic answer exactly at every voxel.

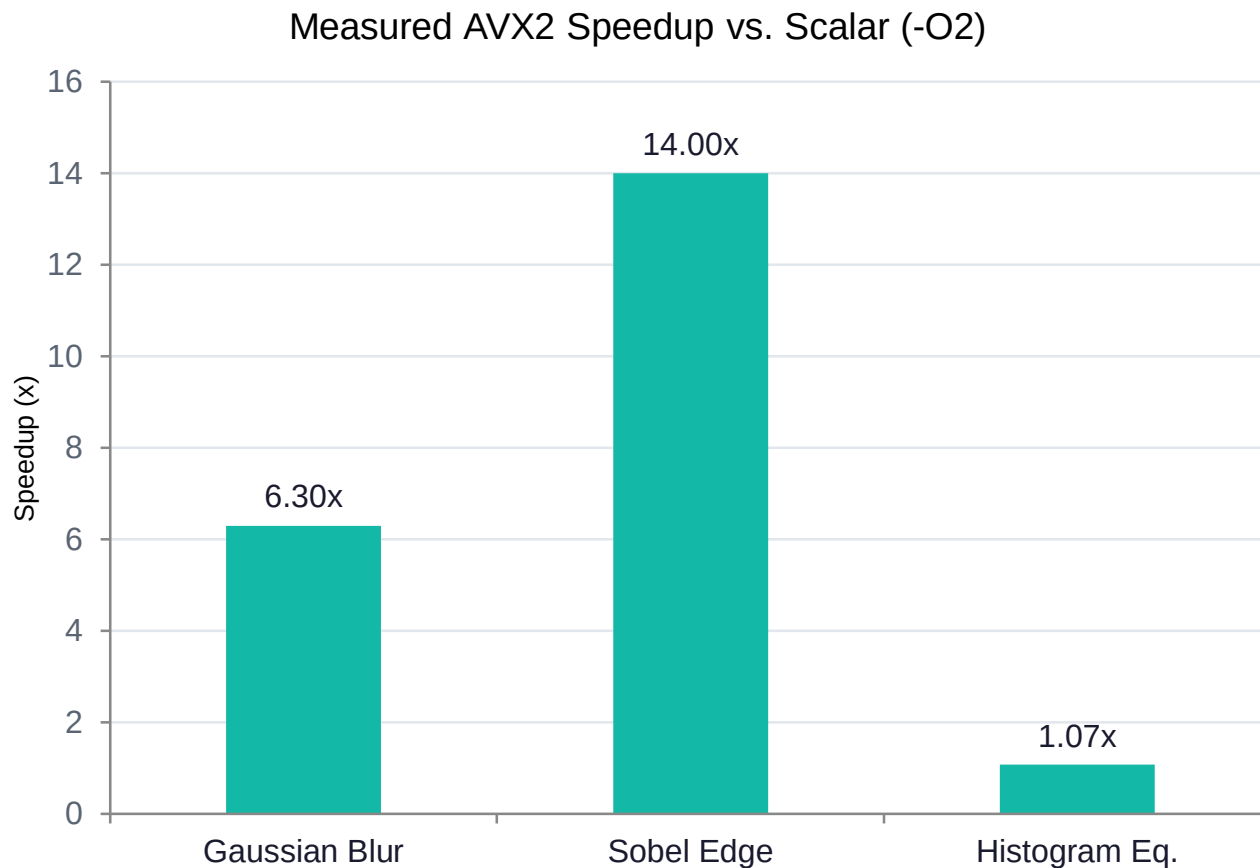
Parallelized (Week 7)

Chunked by output Z-slice across the thread pool. Verified bit-identical to the serial result across 1/2/4/8 threads, and race-free under ThreadSanitizer.

Layer 3: Processing



Hand-written AVX2 SIMD filters, each with a scalar fallback and runtime dispatch



- Interior pixels vectorized 8-wide via `__m256` + FMA
- Border pixels fall back to scalar (per-lane clamping isn't safely vectorizable)
- Histogram remap uses `_mm256_i32gather_ps` against a scalar-built CDF table
- AVX2 flags scoped to exactly one file — an earlier library-wide flag caused SIGILL on non-AVX2 hardware

Layer 4: Detection

Region growing and connected component labeling in one pass



One Algorithm, Two Names

A voxel within the HU threshold is a valid seed. Flood-filling it via 26-connectivity and marking every visited voxel is exactly what connected component labeling does — one pass does both.

Parallel Strategy

- Partition the volume into Z-slabs, flood-fill each independently
- Merge components crossing slab boundaries via union-find — touches only boundary planes, not the whole volume

Ground-Truth Test: Planted Sphere

✓	Centroid	Exact match to planted location
✓	Voxel count	Within 4% of analytic $4/3\pi r^3$
✓	Mean / stddev HU	Exact match on uniform sphere
✓	Serial vs. parallel	Identical results, 7 thread counts
✓	Boundary-straddling case	Explicitly tested — still exact

The Thread Pool



Built from primitives, not `std::async` — `std::thread`, `std::mutex`, `std::condition_variable`, `std::future`

`class ThreadPool`

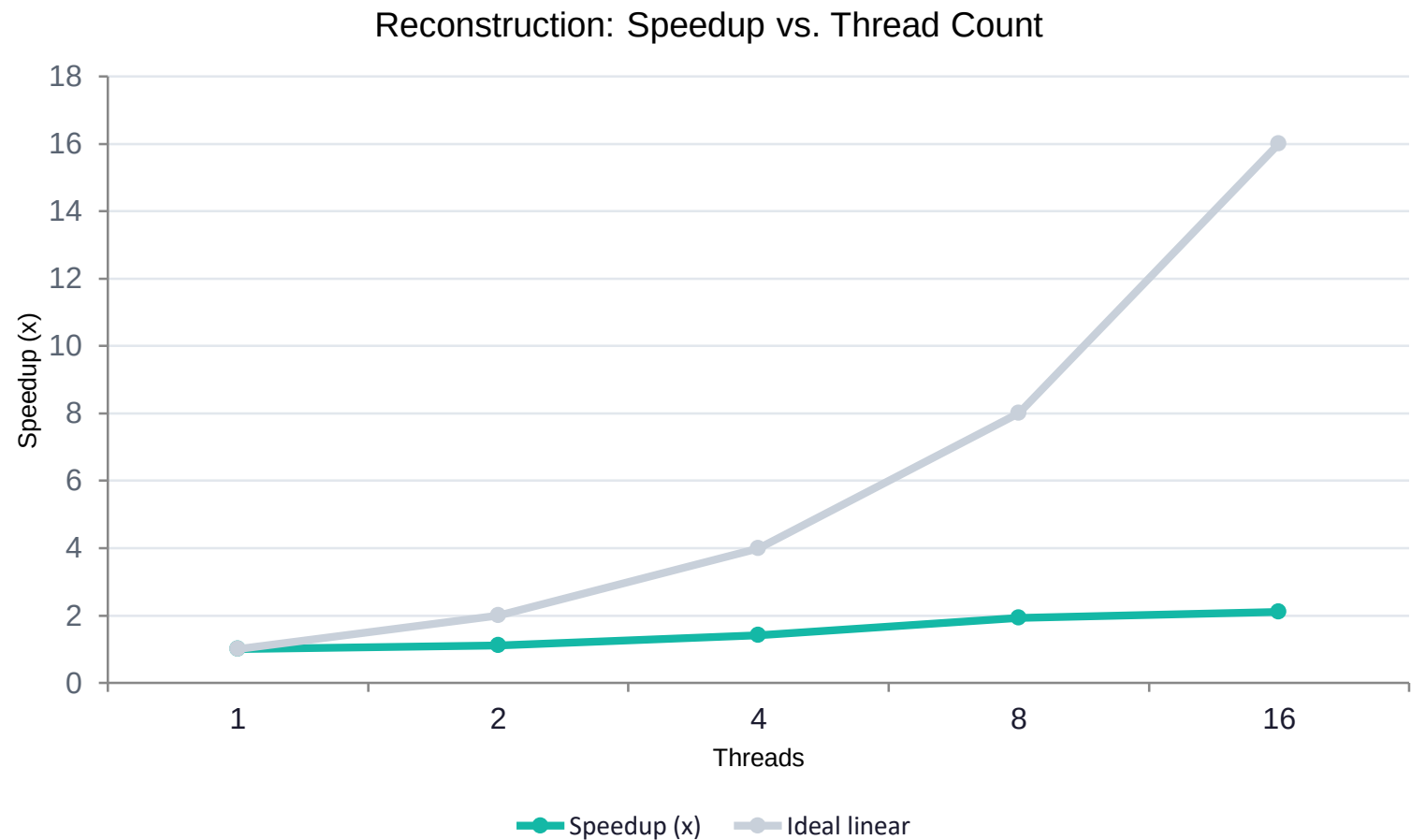
```
ThreadPool(size_t n = 0);  
~ThreadPool(); // drains queue,  
               // then joins  
  
template<class F, class... Args>  
auto submit(F&&, Args&&...)   
    -> future<result_t<F,Args...>>;  
  
size_t threadCount() const;
```

Correctness, Not Just Speed

- 8-test suite incl. a 10,000-task concurrency stress test
- Exception propagation through `future.get()` verified
- Destructor drains all queued work before joining
- Race-free under ThreadSanitizer — including one investigated & documented libstdc++ false positive, not silently ignored
- Applied to two real workloads: Reconstruction and Detection, not a synthetic toy benchmark

Measured Speedup: Reconstruction

Real hardware — Dell Latitude E6510, 8 logical threads, 5-iteration average



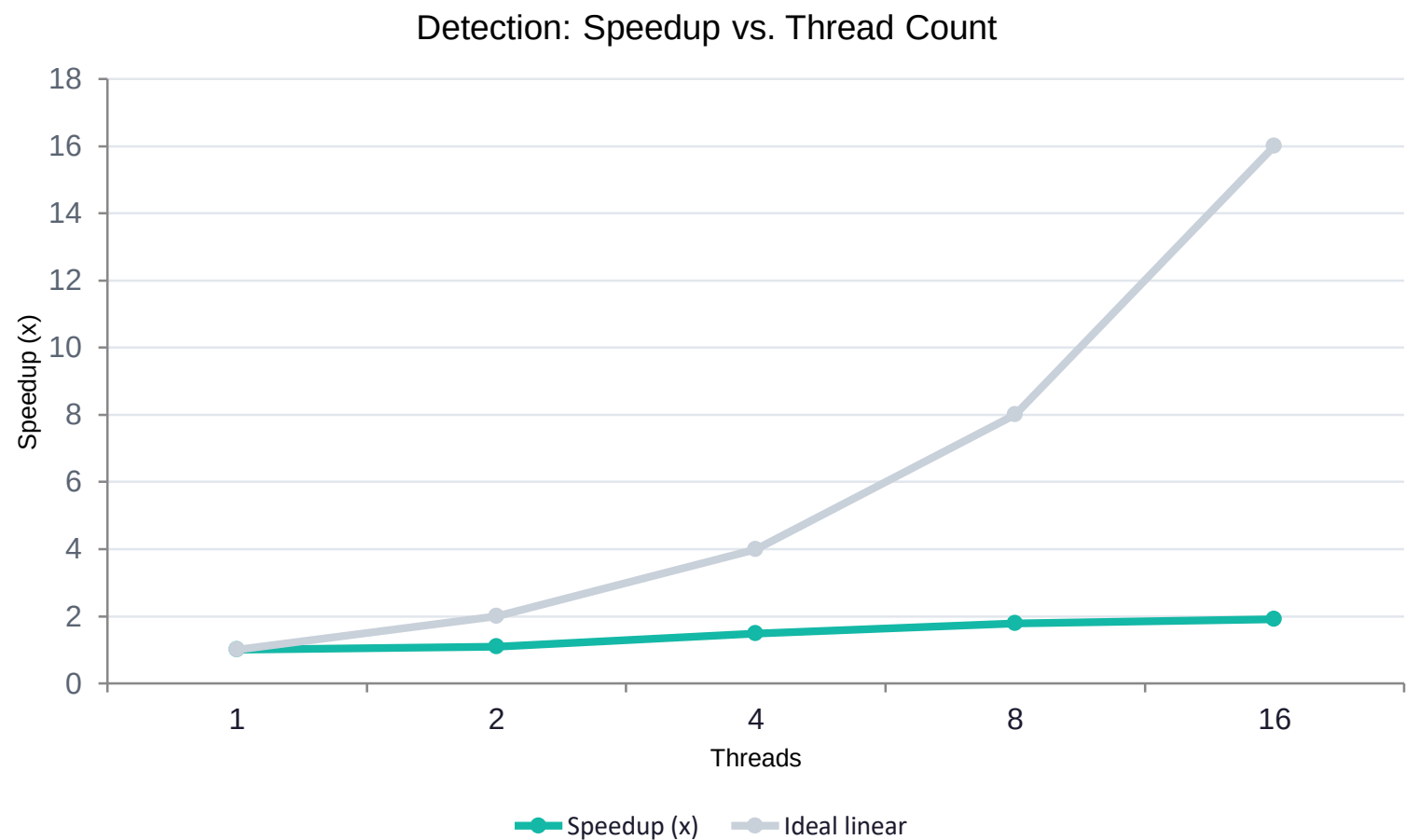
Reading This Honestly

Speedup plateaus around 2x, not 8x. This workload is memory-bandwidth-bound — scattered reads dominate arithmetic, so throughput caps on how fast the CPU is fed data, not core count.

Correctness was verified independently of these numbers. A sub-linear but verified-correct result is a legitimate finding, not a shortfall.

Measured Speedup: Detection

128³ volume, 27 planted nodules, 5-iteration average



Same Pattern, Same Reason

Flood-fill jumps around 3D neighbor indices with poor cache locality — another memory-bound workload, consistent with the Reconstruction result.

Parallel output matched serial exactly at every thread count tested, including a nodule deliberately placed to straddle a slab boundary.

Layer 5: Output

One command produces four artifact types



Annotated PNG

Per-slice grayscale export, HU-windowed, findings outlined in red

Annotated DICOM

Fresh Explicit VR LE file, findings embedded in a private tag — validated against pydicom, an independent library

JSON Findings Report

Structured: series info, volume dimensions, per-finding centroid/bbox/HU stats

HL7 FHIR Bundle

DiagnosticReport + Observation resources, for EHR-integration prototyping

Correctness Before Speed

Every layer verified against a known-correct reference before the next layer depended on it



Ingestion

Fixtures across 3 transfer syntaxes; signed-pixel bug caught & fixed



Reconstruction

Analytic linear-gradient check; bit-identical parallel output



Processing

AVX2-vs-scalar diff on non-multiple-of-8 volumes



Detection

Planted-sphere ground truth; boundary-straddling parallel test



Concurrency

10,000-task stress test; ThreadSanitizer-verified



Output

JSON/FHIR well-formedness; third-party (pydicom) DICOM validation

What's Next

Straightforward extensions of patterns already proven in this codebase



Thread-pool-parallel filters

Apply the same per-Z-slice pattern used in Reconstruction/Detection to the AVX2 filter layer



Broader clinical validation

Run against a larger corpus of real (de-identified) multi-slice series



FHIR conformance

Move from structurally-valid FHIR to a certified clinical profile



GPU acceleration

Trilinear resampling and flood fill are both strong CUDA/compute-shader candidates



Thank You

Pulse Imaging Engine — Questions & Live Demo

Full source, architecture document, and test suite available in the project repository